Cost-Efficient Salient Object Segmentation

A 176K-parameter residual U-Net under a 50 ms single-core CPU budget

Prasangeet Dongre (Roll No. B23CH1033)
B.Tech Chemical Engineering, IIT Jodhpur • CSL7360 Computer Vision, Assignment 2 (Track B)

0.749	0.519	0.680	17.3 ms	0.73 MiB	0.6205
mIoU (DUTS-TE)	Boundary F1	quality Q	T_{CPU} per image	S_{disk}	CES

Abstract. We build a binary salient-object segmenter that has to label every pixel of a 256×256 photograph within 50 ms of single-thread CPU time and a 4 MiB parameter file. Our model is a three-level residual U-Net with a dilated depthwise multi-scale context block at the bottleneck and deep supervision, trained from scratch on DUTS-TR at 128×128 working resolution. On the full DUTS-TE benchmark (5,019 images) it reaches mIoU = 0.7491, Boundary F1 = 0.5192 and $Q = 0.6801$ at $T_{\text{CPU}} = 17.32$ ms with a 767,807-byte checkpoint, giving CES = 0.6205. On seven hand-annotated photographs from our own surroundings it reaches mIoU = 0.8193. The model is cheap enough that the remaining weakness is not speed but contour accuracy and clutter handling, which we analyse through three failure cases.

1 Introduction

The assignment asks for a pipeline that separates the main object from the background, scored by

$$Q = 0.70\,\text{mIoU} + 0.30\,\text{BF1}, \tag{1}$$

$$\text{CES} = Q\Big(1 - 0.20\,\tfrac{T_{\text{CPU}}}{50\,\text{ms}} - 0.10\,\min\big(1, \tfrac{S_{\text{disk}}}{4\,\text{MiB}}\big)\Big), \tag{2}$$

with CES set to zero if $T_{\text{CPU}} > 50$ ms. We follow *Track B* (a small fully convolutional network written and trained from scratch). The design question is therefore twofold: how accurate can a network be, and how little of the time and storage budget does it need to spend to get there? Both penalty terms are linear, so each millisecond costs 0.4% of Q and the whole file-size term at most 10%. A model that is accurate *and* tiny wins over a bigger, slightly more accurate one.

2 Data and Preprocessing

2.1 Splits

DUTS-TR (10,553 images) is split into a training and a validation part; DUTS-TE (5,019 images) is the final benchmark. The validation set is 10% of DUTS-TR, drawn deterministically (seed 42) and stratified by foreground area: images are sorted by the fraction of mask pixels that are object, cut into strata, and sampled within each stratum, so that small and large objects appear in both parts in similar proportions.

Split	Images	Use
DUTS-TR (train)	9,498	gradient updates
DUTS-TR (val)	1,055	monitoring, checkpoint choice
DUTS-TE	5,019	final benchmark
Own photos	7	test only

Table 1. Dataset splits.

Test-set hygiene. DUTS-TE is never loaded by the training loop, never contributes gradients and is not used to pick the saved checkpoint. In full disclosure, however, DUTS-TE scores were looked at while comparing candidate designs during development, so it is not a strictly blind set. The reported figure is the single final run under the protocol below.

2.2 Preprocessing and augmentation

Following the assignment, every photo is resized to 256×256 with `cv2.INTER_AREA` and every mask with `cv2.INTER_NEAREST`; a mask pixel is object if its value is at least 128. All metrics are computed on these 256×256 masks. Inputs are RGB floats in $[0, 1]$. During training only, image and mask receive the same random horizontal flip (p=0.5), vertical flip (p=0.10) and isotropic rescaling by 0.9–1.1 (p=0.5, new border pixels are background); the image alone additionally gets brightness and per-channel gain jitter with clipping.

3 Method

3.1 Architecture

The network (Fig. 1) is a three-level residual U-Net with 175,971 trainable parameters. Each encoder block has two 3×3 convolutions with batch norm and ReLU plus a residual shortcut (projected with a 1×1 convolution when channel counts differ). Channel widths are kept very small, $16 \to 24 \to 40 \to 56$, and max pooling halves the resolution between stages.

Dilated multi-scale context. Salient objects range from a computer mouse to a whole backpack, so the bottleneck (at 1/8 of the working resolution) needs several receptive-field sizes. Three parallel branches, each a 1×1 projection to 16 channels followed by a *depthwise* 3×3 convolution with dilation 1, 2 and 4, are concatenated, projected back to 56 channels and added to the bottleneck input. Depthwise convolutions give the wider context for a fraction of the parameters and multiply–adds of a full dilated block.

Decoder and skips. The decoder upsamples, concatenates the matching encoder feature (U-Net style skip connections) and applies a residual block: $56{+}40 \to 40$, $40{+}24 \to 24$, $24{+}16 \to 16$. A final $1{\times}1$ convolution gives one logit per pixel. Two auxiliary $1{\times}1$ heads on the quarter- and half-resolution decoder stages exist *only during training*; at inference only the main head runs.

3.2 Loss and training

Every supervised output uses an equal mix of binary cross-entropy (with logits) and soft Dice loss,

$$\mathcal{L}_{\text{base}} = 0.5\,\mathcal{L}_{\text{BCE}} + 0.5\,\mathcal{L}_{\text{Dice}}, \tag{3}$$

and deep supervision weights the main and auxiliary outputs as

$$\mathcal{L} = 0.60\,\mathcal{L}_{\text{main}} + 0.15\,\mathcal{L}_{1/4} + 0.25\,\mathcal{L}_{1/2}. \tag{4}$$

BCE gives stable per-pixel gradients, while Dice counteracts the strong background majority that makes plain cross-entropy under-segment small objects.

We train from scratch (no pretrained weights) with AdamW for 60 epochs on a GPU, batch size 8 with 2 accumulation steps (effective 16), peak learning rate $7{\times}10^{-4}$, 3 epochs of linear warm-up followed by cosine decay to 10^{-6}, weight decay 10^{-5} (not applied to biases and normalisation parameters), gradient clipping at 1.0 and mixed precision. The checkpoint is chosen by validation quality $Q_{\text{val}} = 0.70\,\text{mIoU} + 0.30\,\text{BF1}$, which peaked at epoch 56 (Fig. 2).

4 Keeping the Pipeline Fast

The timed region starts with a $256{\times}256$ BGR photograph in memory and ends with a $256{\times}256$ array of 0s and 1s. Inside it we:

1. downscale to $128{\times}128$ with `INTER_AREA`,
2. convert BGR$\to$RGB, scale to $[0, 1]$ and build the tensor,
3. run the network,
4. bilinearly upsample the logits to $256{\times}256$,
5. apply a sigmoid, threshold at 0.5 and materialise the NumPy mask.

File I/O, resizing the original photograph to $256{\times}256$, loading the checkpoint and metric computation lie outside the timer. The main levers on cost were:

- **Half working resolution.** Running the CNN at $128{\times}128$ rather than $256{\times}256$ cuts the number of pixels processed by $4{\times}$; the logits are upsampled afterwards.
- **Very narrow channels** (16–56) and a context block built from depthwise convolutions.
- **No test-time extras:** no ensembling, no flip averaging, no auxiliary heads at inference.
- **Strict single-thread setup.** The four thread environment variables are set to `"1"` before importing NumPy, OpenCV or PyTorch, followed by `cv2.setNumThreads(1)` and `torch.set_num_threads(1)`; the model is warmed up once before timing.

T_{CPU} is the total `time.process_time()` over all 5,019 DUTS-TE images divided by 5,019. The storage side is equally simple: the saved file is a plain, uncompressed PyTorch state dict of 767,807 bytes (0.7322 MiB, 18.3% of the 4 MiB budget), all learned values live in it, and none are hard-coded.

Evaluation code. Metrics are written from scratch in NumPy. mIoU accumulates one $2{\times}2$ confusion matrix over the set. For Boundary F1, boundary pixels are found with four shifted copies of the padded mask (pixels outside the image count as background), and the $5{\times}5$ tolerance check is a logical OR over 25 shifted slices of the boundary map, so no per-pixel Python loops or prohibited morphology calls are needed.

5 Results

Hardware. All timings were measured on CPU only, with one thread, using PyTorch and OpenCV. CPU: **[CPU model: 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz]**.

5.1 DUTS-TE benchmark

Table 2 summarises all three evaluation sets. On DUTS-TE the pipeline spends 17.32 ms per image (34.6% of the time budget), so both hard limits are met with room to spare and the efficiency penalty is only

$$\left(1 - 0.20 \cdot \tfrac{17.32}{50} - 0.10 \cdot \tfrac{767807}{4194304}\right) = 0.9124.$$

Hence CES $= 0.6801 \times 0.9124 = 0.6205$.

Background IoU (0.9106) is far higher than foreground IoU (0.5875): the aggregate confusion matrix shows 14.5M background pixels wrongly marked as object but also 11.6M object pixels missed, against 37.1M correctly found. Boundary F1 of 0.52 confirms that contours, more than regions, are the main weakness. Qualitative DUTS-TE examples are in Fig. 4: compact, high-contrast objects (butterfly, fawn) are segmented well, while large objects in clutter (the seated man) and multi-part scenes are only partially recovered.

Validation-to-test gap. Validation Q (0.853) is much higher than test Q (0.680). Part of this is the resolution mismatch noted in Table 2, and part is likely that the validation split, drawn from the same distribution as the training data, is easier than the official test set; the gap should not be read as a clean estimate of overfitting.

5.2 Own photographs

We photographed seven everyday objects and annotated them by hand on the $256{\times}256$ resized images (object $= 255$, background $= 0$). The set deliberately includes cluttered backgrounds, partial occlusion, a small object and a thin-structure case. Fig. 3 shows ground-truth and predicted masks as semi-transparent overlays and

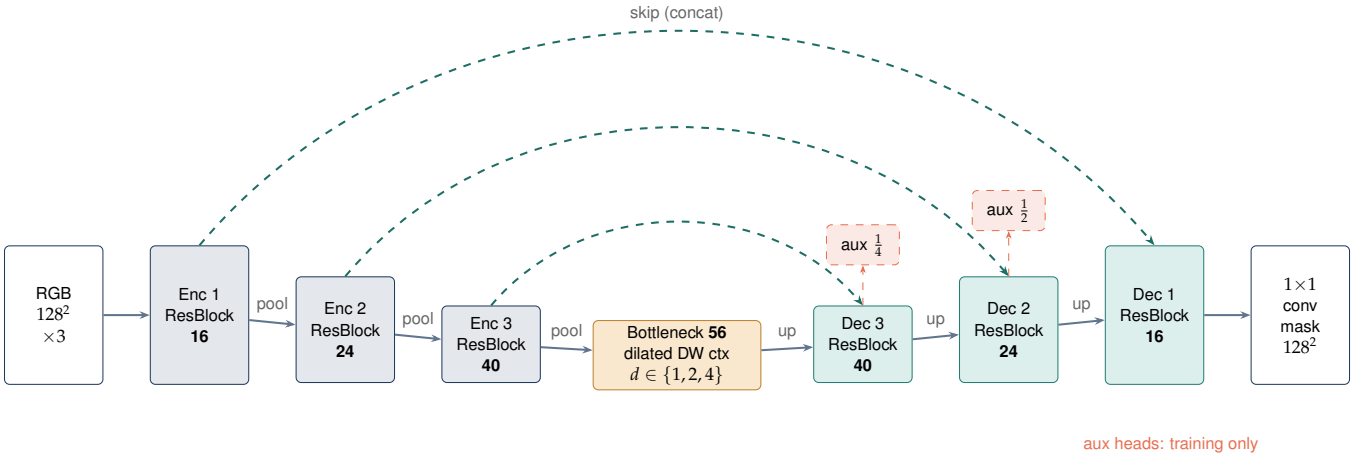


Figure 1. Architecture of the final network (175,971 parameters). Numbers in bold are channel counts. The dashed teal arcs are U-Net skip connections that concatenate encoder features into the decoder; the coral heads provide deep supervision during training and are removed at inference.

Set	Images	Res.	mIoU	BF1	Q	T_{CPU} (ms)	S_{disk} (bytes)	CES
Validation (DUTS-TR split)	1,055	128^2[a]	0.8819	0.7854	0.8529	–	767,807	–
DUTS-TE (official)	5,019	256^2	**0.7491**	**0.5192**	**0.6801**	**17.32**	**767,807**	**0.6205**
Own photos	7	256^2	0.8193	0.5629	0.7424	17.06[b]	767,807	–

Table 2. Main results. mIoU is computed from one aggregate confusion matrix; BF1 is the mean of per-image scores. [a]Validation numbers come from the 128×128 training loop and are therefore not resolution-matched with the 256×256 test numbers. [b]Mean over 140 timed predictions (20 per photo after a warm-up); the official T_{CPU} is the DUTS-TE average. CES is reported only for DUTS-TE, the common benchmark.

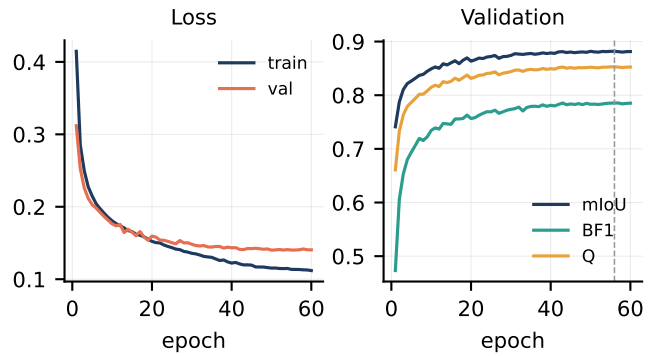


Figure 2. Training history. Left: training and validation loss. Right: validation mIoU, Boundary F1 and Q (dashed line: selected epoch 56).

ID	Object	IoU_{fg}	mIoU	BF1
1	Chair (occluded)	0.867	0.915	0.720
2	Suitcase	0.469	0.579	0.298
3	Two bicycles	0.747	0.809	0.445
4	Backpack	0.947	0.938	0.651
5	Pair of clogs	0.922	0.944	0.867
6	Game controller	0.924	0.950	0.795
7	Computer mouse	0.503	0.648	0.165
Aggregate		0.759	0.819	0.563[c]

Table 3. Per-photo results. [c]Mean per-image BF1; mean per-image mIoU is 0.826.

Table 3 gives per-image scores. These photographs were used only for testing.

Timing over 140 predictions averaged 17.06 ms (median 16.61 ms, std. 1.83 ms, min 14.63 ms, max 21.60 ms), consistent with the DUTS-TE figure.

6 Failure Analysis

Suitcase (ID 2): occlusion and internal detail. Foreground IoU is only 0.469 and BF1 0.298. The suitcase is partly covered by books and cloth, its surface is a uniform beige that resembles the floor and wall, and dark seams and wheels create strong internal edges. The prediction therefore contains holes inside the object and leaks into the cloth on top; with only a 128×128 working resolution and a 176K-parameter model there is little capacity to learn that a plain surface can still belong to the salient object.

Computer mouse (ID 7): small object, textured and glowing background. BF1 is just 0.165 (mIoU 0.648). The mouse is small, dark and lit by coloured light, sitting on a high-contrast geometric mousepad next to a cable and a second object. The network labels part of the mousepad as foreground and puts the contour in the wrong place, so a boundary that is a few pixels off is penalised heavily by the 5×5 tolerance. A small object at 128×128 has only a handful of pixels per boundary segment, which also hurts.



Figure 3. Own photographs: input, ground-truth overlay (teal) and predicted overlay (coral), with per-image mIoU. Seven photos, all kept in the evaluation, including the difficult ones.

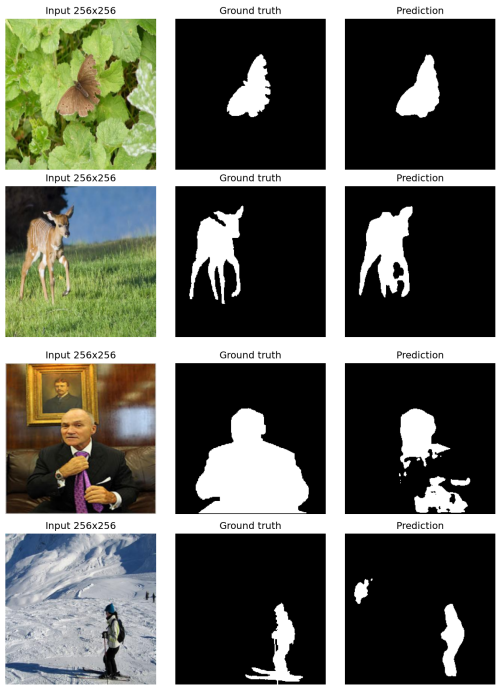


Figure 4. DUTS-TE examples (input, ground truth, prediction). Good: butterfly, fawn. Failures: the seated man (large object, dark clothing blending with the sofa) and the skier (small object, spurious blob on the snow).

Two bicycles (ID 3): thin structures. Region overlap is respectable (mIoU 0.809), but BF1 drops to 0.445. Spokes, frames and the overlap between the two bikes are thin, and the prediction merges them into a smoother blob that fills the spaces between the wheels. This shows why region overlap alone is not enough to judge segmentation quality.

Caveats. Image 1 (chair under clothes) scores high because its target was defined as the visible chair-plus-clothes region; it should not be read as evidence that the model segments chairs in general. With seven photographs and hand-drawn masks, per-image scores carry annotation uncertainty, and none of these numbers is a statistically meaningful estimate of population performance.

7 Discussion and Future Work

The model achieves its goal as an efficiency trade-off: 17.3 ms and 0.73 MiB leave most of both budgets unused, and CES is reduced by only 8.8% relative to Q. Since the budget is so far from binding, the most promising next steps spend some of it on quality:

- raise the working resolution (e.g. 160 or 192) and measure the gain against the 0.4%-per-millisecond cost;
- add boundary-aware supervision (e.g. an edge loss), aimed directly at the weak BF1 term;
- strengthen multi-scale fusion and add a stronger context block for large cluttered objects;
- evaluate on a larger, separately annotated hard-scene set, and take every design decision on validation data only.

Reproducibility. Code, the final checkpoint `final/unet_128_multiscale_v1/model_best.pt`, evaluation scripts and the captured photographs (`captured/{original, images_256, masks_256}`) are in the project repository. `predict.py -image photo.jpg -out mask.png` resizes the photo, segments it, saves a 0/255 PNG and prints `T CPU ms` and `S disk bytes` averaged over 20 further runs. The result files are `results_duts-te_256.json` and `results_captured.json`.